5. 클라이언트 통합 프로세스

개요

Sapphire ParaTime과 기존 Ethereum 생태계 도구들의 통합 과정과

개발자들이 기밀 dApp을 쉽게 개발할 수 있도록 제공되는

클라이언트 라이브러리 및 통합 도구들을 상세히 설명합니다.

5.1 클라이언트 아키텍처

다중 언어 지원

```
┌──────────────────────────────────────────────────────────────┐
│                        Developer Tools                       │
│ ┌──────────────────────────────────────────────────────────┐ │
│ │                   Frontend Integration                   │ │
│ │ ┌──────────────────────────────────────────────────────┐ │ │
│ │ │                TypeScript/JavaScript                 │ │ │
│ │ │        - @oasisprotocol/sapphire-paratime            │ │ │
│ │ │        - @oasisprotocol/sapphire-ethers-v6           │ │ │
│ │ │        - @oasisprotocol/sapphire-wagmi-v2            │ │ │
│ │ │        - @oasisprotocol/sapphire-viem-v2             │ │ │
│ │ └──────────────────────────────────────────────────────┘ │ │
```

```
│ │ ┌──────────────────────────────────────────────┐ │ │
│ │ │                Backend Integration           │ │ │
│ │ │                 - Go Client                  │ │ │
│ │ │                 - Python Client              │ │ │
│ │ └──────────────────────────────────────────────┘ │ │
│ └──────────────────────────────────────────────────┘ │
│ ┌──────────────────────────────────────────────────┐ │
│ │                  Development Tools               │ │
│ │          - @oasisprotocol/sapphire-hardhat       │ │
│ │          - @oasisprotocol/sapphire-contracts     │ │
│ └──────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────┘
```

패키지 생태계

| 패키지 | 버전 | 용도 | 다운로드 |

|--------|------|------|----------|

| sapphire-paratime | npm v2.2.0 | 핵심 TypeScript 클라이언트 | downloads 1.8k/month |

| sapphire-hardhat | npm v2.22.2 | Hardhat 통합 플러그인 | downloads 314/month |

| sapphire-contracts | npm v0.2.14 | Solidity 라이브러리 | downloads 320/month |

| sapphire-ethers-v6 | npm v6.0.1 | Ethers.js v6 통합 | downloads 212/month |

5.2 TypeScript/JavaScript 통합

기본 클라이언트 사용법

```
import { Sapphire } from '@oasisprotocol/sapphire-paratime';
import { ethers } from 'ethers';

// Sapphire 네트워크 연결
const provider = Sapphire.wrap(
new ethers.providers.JsonRpcProvider('https://sapphire.oasis.io')
);

// 지갑 연결
const wallet = new ethers.Wallet(privateKey, provider);

// 기밀 트랜잭션 전송
const contract = new ethers.Contract(contractAddress, abi, wallet);
const tx = await contract.confidentialFunction(encryptedData);
```

자동 암호화 래퍼

```
// 자동 calldata 암호화
const wrappedProvider = Sapphire.wrap(provider);
const wrappedWallet = Sapphire.wrap(wallet);
```

```
// 모든 트랜잭션이 자동으로 암호화됨
const result = await contract.connect(wrappedWallet).someFunction(data);
```

5.3 Ethers.js 통합

설치 및 설정

```
npm install @oasisprotocol/sapphire-ethers-v6 ethers@^6
```

사용 예시

```
import { Sapphire } from '@oasisprotocol/sapphire-ethers-v6';
import { ethers } from 'ethers';

// 기존 Ethers.js 코드를 Sapphire로 래핑
const provider = Sapphire.wrap(
new ethers.JsonRpcProvider('https://sapphire.oasis.io')
);

const wallet = Sapphire.wrap(
new ethers.Wallet(process.env.PRIVATE_KEY!, provider)
);

// 기밀 컨트랙트 배포
```

```
const contractFactory = new ethers.ContractFactory(abi, bytecode, wallet);
const contract = await contractFactory.deploy();
await contract.waitForDeployment();
```

5.4 Wagmi 통합

React 애플리케이션 통합

```
import { Sapphire } from '@oasisprotocol/sapphire-wagmi-v2';
import { createConfig, http } from 'wagmi';

// Sapphire 네트워크 설정
const config = createConfig({
chains: [Sapphire.chains.sapphire],
transports: {
[Sapphire.chains.sapphire.id]: Sapphire.transport(
http('https://sapphire.oasis.io')
),
},
});

// React 컴포넌트에서 사용
function MyComponent() {
```

```
const { data: account } = useAccount();
const { writeContract } = useWriteContract();


const handleConfidentialCall = async () => {
writeContract({
address: contractAddress,
abi: contractAbi,
functionName: 'confidentialFunction',
args: [encryptedData],
});
};


return <button onClick={handleConfidentialCall}>Send Confidential TX</button>;
}
```

5.5 Viem 통합

저수준 클라이언트 통합

```
import { Sapphire } from '@oasisprotocol/sapphire-viem-v2';
import { createWalletClient, createPublicClient, http } from 'viem';


// 퍼블릭 클라이언트 생성
```

```
const publicClient = Sapphire.wrapPublicClient(
createPublicClient({
chain: Sapphire.chains.sapphire,
transport: http('https://sapphire.oasis.io'),
})
);

// 지갑 클라이언트 생성
const walletClient = Sapphire.wrapWalletClient(
createWalletClient({
chain: Sapphire.chains.sapphire,
transport: http('https://sapphire.oasis.io'),
account: privateKeyToAccount(process.env.PRIVATE_KEY!),
})
);

// 기밀 트랜잭션 실행
const hash = await walletClient.writeContract({
address: contractAddress,
abi: contractAbi,
functionName: 'confidentialFunction',
args: [data],
```

```
});
```

5.6 Hardhat 통합

프로젝트 설정

```
npm install @oasisprotocol/sapphire-hardhat hardhat
```

hardhat.config.ts 설정

```
import '@oasisprotocol/sapphire-hardhat';
import { HardhatUserConfig } from 'hardhat/config';

const config: HardhatUserConfig = {
solidity: '0.8.19',
networks: {
sapphire: {
url: 'https://sapphire.oasis.io',
accounts: [process.env.PRIVATE_KEY!],
chainId: 0x5afe,
},
'sapphire-testnet': {
url: 'https://testnet.sapphire.oasis.io',
accounts: [process.env.PRIVATE_KEY!],
```

```
chainId: 0x5aff,
},
'sapphire-localnet': {
url: 'http://localhost:8545',
accounts: [process.env.PRIVATE_KEY!],
chainId: 0x5afd,
},
},
};

export default config;
```

배포 스크립트

```
import { ethers } from 'hardhat';
import { Sapphire } from '@oasisprotocol/sapphire-hardhat';

async function main() {
// Sapphire 래퍼 적용
const [deployer] = await ethers.getSigners();
const wrappedDeployer = Sapphire.wrap(deployer);

// 기밀 컨트랙트 배포
```

```
const ConfidentialContract = await ethers.getContractFactory(
'ConfidentialContract',
wrappedDeployer
);
const contract = await ConfidentialContract.deploy();
await contract.waitForDeployment();

console.log('Contract deployed to:', await contract.getAddress());
}

main().catch(console.error);
```

5.7 Go 클라이언트 통합

설치 및 기본 사용법

```
go get github.com/oasisprotocol/sapphire-paratime/clients/go

package main

import (
"context"
"crypto/ecdsa"
```

```
"math/big"
"github.com/ethereum/go-ethereum/crypto"
"github.com/ethereum/go-ethereum/ethclient"
sapphire  "github.com/oasisprotocol/sapphire-paratime/clients/go"
)

func  main() {
// Ethereum 클라이언트 생성
client, err := ethclient.Dial("https://sapphire.oasis.io")
if  err != nil {
panic(err)
}

// Sapphire 래퍼 적용
sapphireClient := sapphire.WrapClient(client)

// 개인키 로드
privateKey, err := crypto.HexToECDSA("your-private-key")
if  err != nil {
panic(err)
}

// 기밀 트랜잭션 생성 및 전송
```

```
tx, err := sapphireClient.SendConfidentialTransaction(
context.Background(),
privateKey,
contractAddress,
calldata,
big.NewInt(0), // value
)
if  err != nil {
panic(err)
}


fmt.Printf("Transaction hash: %s\n", tx.Hash().Hex())
}
```

5.8 Python 클라이언트 통합

Web3.py 통합

```
from web3 import Web3
from sapphire_paratime import SapphireMiddleware


# Web3 인스턴스 생성
w3 = Web3(Web3.HTTPProvider('https://sapphire.oasis.io'))
```

```
# Sapphire 미들웨어 추가
w3.middleware_onion.add(SapphireMiddleware)


# 계정 설정
account = w3.eth.account.from_key('your-private-key')
w3.eth.default_account = account.address


# 기밀 트랜잭션 전송
contract = w3.eth.contract(address=contract_address, abi=contract_abi)
tx_hash = contract.functions.confidentialFunction(data).transact()


# 트랜잭션 확인
receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
print(f"Transaction confirmed: {receipt.transactionHash.hex()}")
```

5.9 개발 도구 및 유틸리티

로컬 개발 환경

```
# Sapphire Localnet 실행
docker run -it -p 8545:8545 -p 8546:8546 \
ghcr.io/oasisprotocol/sapphire-localnet:latest
```

```
# 또는 개발용 노드 직접 실행
make debug
```

테스트 도구

```
// Hardhat 테스트에서 Sapphire 사용
import { expect } from 'chai';
import { ethers } from 'hardhat';
import { Sapphire } from '@oasisprotocol/sapphire-hardhat';

describe('ConfidentialContract', function () {
it('should handle confidential data', async function () {
const [owner] = await ethers.getSigners();
const wrappedOwner = Sapphire.wrap(owner);

const Contract = await ethers.getContractFactory('ConfidentialContract');
const contract = await Contract.connect(wrappedOwner).deploy();

// 기밀 데이터로 테스트
const secretData = ethers.utils.toUtf8Bytes('secret message');
await expect(contract.storeSecret(secretData)).to.not.be.reverted;
});
```

```
});
```

5.10 마이그레이션 가이드

기존 Ethereum dApp 마이그레이션

1. **의존성 추가**

```
npm install @oasisprotocol/sapphire-paratime
```

2. **Provider 래핑**

```
// 기존 코드
const provider = new ethers.providers.JsonRpcProvider(rpcUrl);
// Sapphire 적용
const provider = Sapphire.wrap(
new ethers.providers.JsonRpcProvider(sapphireRpcUrl)
);
```

3. **네트워크 설정 변경**

```
const networks = {
sapphire: {
chainId: 0x5afe,
rpcUrl: 'https://sapphire.oasis.io',
```

```
},
'sapphire-testnet': {
chainId: 0x5aff,
rpcUrl: 'https://testnet.sapphire.oasis.io',
},
};
```

점진적 마이그레이션

1. **하이브리드 접근**

- 민감한 기능만 Sapphire로 이전
- 기존 Ethereum 컨트랙트와 상호작용 유지
- 크로스체인 브리지 활용

2. **데이터 마이그레이션**

- 기존 상태를 암호화하여 Sapphire로 이전
- 점진적 사용자 마이그레이션
- 백워드 호환성 유지

5.11 성능 최적화

클라이언트 최적화

```
// 배치 요청으로 성능 향상
const multicall = new ethers.Contract(multicallAddress, multicallAbi, prov
const calls = [
```

```
contract.interface.encodeFunctionData('function1', [param1]),
contract.interface.encodeFunctionData('function2', [param2]),
];

const results = await multicall.aggregate(calls);
```

캐싱 전략

```
// 결과 캐싱
const cache = new Map();

async function cachedCall(key: string, fn: () => Promise<any>) {
if (cache.has(key)) {
return cache.get(key);
}
const result = await fn();
cache.set(key, result);
return result;
}
```

결론

Sapphire의 클라이언트 통합 프로세스는 기존 Ethereum 개발 도구들과의

완벽한 호환성을 제공하면서도 기밀성 기능을 투명하게 추가합니다.

개발자들은 최소한의 코드 변경으로 기존 dApp을 기밀 dApp으로

전환할 수 있으며, 다양한 언어와 프레임워크를 지원하여

광범위한 개발 생태계를 포괄합니다.